

VISTA: Verifier-in-the-Loop Agentic RL for Semantic Program Synthesis in Quantum Computing

Submission ID

Abstract

We present VISTA, a verifier-in-the-loop, tool-augmented agentic RL system for semantic program synthesis, with OpenQASM 3.0 quantum circuit generation as a successful application. Our core idea is verifier-in-the-loop post-training: an external quantum verification toolchain evaluates complete generated programs and returns training signals grounded in execution semantics rather than text plausibility. To make this practical and stable, VISTA introduces two optimization techniques: (i) hierarchical verified reward optimization, which stages learning from executability to distributional alignment, objective-level semantic quality, and optimization utility; and (ii) cost-aware gated evaluation optimization, which prioritizes low-cost checks and invokes expensive simulator/optimizer evaluations only for promising candidates. Across a broad suite of quantum optimization tasks, VISTA improves both executability and semantic quality over prompting-based industrial LLMs and SFT-only and RL-only post-training baselines. These results show that verifier-in-the-loop agentic RL, combined with hierarchical reward shaping and gated evaluation, is an effective design pattern for AI agent systems in domains with expensive semantic verification, with quantum circuit generation as a successful application.

ACM Reference Format:

Submission ID. 2018. VISTA: Verifier-in-the-Loop Agentic RL for Semantic Program Synthesis in Quantum Computing. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 11 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

LLMs can now write code that *looks* correct; they still fail when correctness is determined by expensive execution-time semantics. This gap is now the bottleneck for deployable AI agents in program synthesis. In semantic domains, plausibility is not utility: outputs must compile, satisfy resource constraints, and optimize task objectives under verification. Quantum assembly generation is a stress test for this regime.

OpenQASM 3.0 sits at the hardware-facing boundary of modern quantum software stacks [2, 7, 9, 18]. It is therefore a high-impact target for natural-language-to-program generation. But the failure modes are stacked and costly: (i) compilation failure, (ii) qubit-count mismatch, and (iii) compilable circuits with semantically poor parameterizations. Prompt-only generation and standard SFT/RL

recipes are brittle here because the reliable signal arrives through expensive external verification rather than text-level likelihood.

This yields a concrete systems question for AI agents: *how do we train a generator when supervision is verifier-grounded, sparse, and non-differentiable?* A useful solution must jointly optimize three objectives: training stability, verifier efficiency, and semantic fidelity. Optimizing only one of them fails in practice—stable but semantics-weak models, semantics-aware but prohibitively expensive loops, or high variance training with weak convergence.

We address this with VISTA, a verifier-in-the-loop, tool-augmented agentic RL system for semantic program synthesis, with OpenQASM 3.0 quantum circuit generation as a successful application. VISTA couples an LLM actor with an external verifier stack (compiler + simulator + task evaluator), and trains policies from verified feedback instead of text plausibility. The key design is two-fold. First, *hierarchical verified reward optimization* stages supervision from cheap high-signal checks (executability) to expensive semantic checks (distributional and objective quality), then to practical optimization utility. Second, *cost-aware gated evaluation optimization* prioritizes low-cost checks and invokes expensive simulator/optimizer evaluation only for promising candidates.

The gains are substantial. At Pass@1, VISTA reaches 99.31% syntactic correctness (SCR), 22.41% semantic success rate (SREV), 11.61 KL-based relative entropy error (RE), and 17.24% high-quality circuit ratio (HQCR). At Pass@10, VISTA attains 100% SCR, 33.10% SREV, 8.48 RE, and 27.24% HQCR. Relative to strong industrial LLM baselines, VISTA improves executability by up to 13.62% and semantic quality by 10.3%–35% across key metrics. These results show that verifier-in-the-loop agentic RL is not only viable but *effective* for domains with expensive semantic verification.

Contributions.

- **Core system contribution.** We design VISTA, a verifier-in-the-loop, tool-augmented agentic RL system that translates natural-language specifications into executable OpenQASM 3.0 programs.
- **Optimization technique I: hierarchical verified reward optimization.** We introduce a staged reward design that optimizes generation from syntax correctness to distributional alignment, objective-level semantic quality, and optimization utility.
- **Optimization technique II: cost-aware gated evaluation optimization.** We propose a gated verifier execution policy that runs expensive simulator/optimizer checks only for promising candidates, improving training efficiency while preserving semantic learning quality.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06

<https://doi.org/XXXXXXX.XXXXXXX>

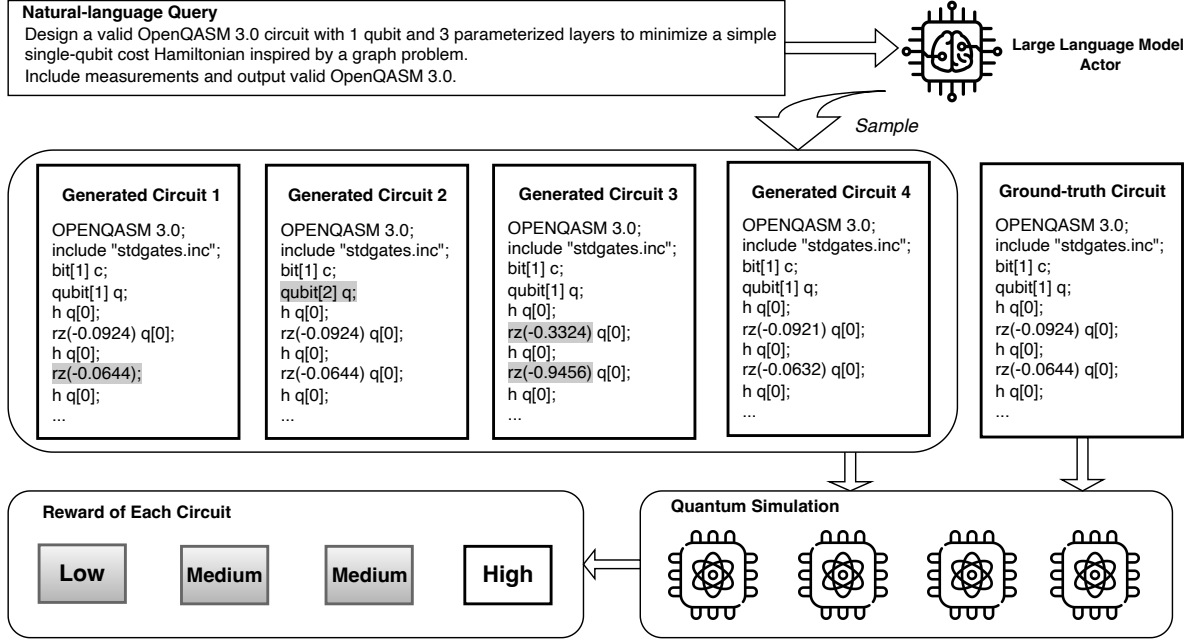


Fig. 1: Illustration of an end-to-end training example in VISTA. Given a natural-language query, the LLM actor samples multiple candidate OpenQASM 3.0 circuits. During verification, sampled circuits may (a) fail to compile, (b) compile but use an incorrect number of qubits, (c) compile with the correct qubit count but suboptimal parameter values, or (d) compile successfully with the correct qubit count and well-aligned parameters. Verified circuits are evaluated via quantum simulation and mapped to scalar rewards, which are then used for policy updates.

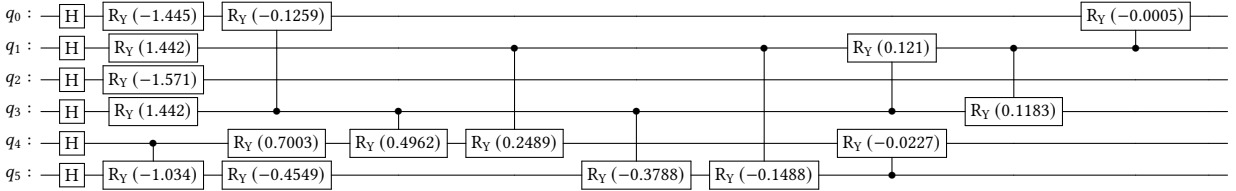


Fig. 2: Example of an LLM-generated ansatz with initial parameters.

2 Background and Problem Context

2.1 Agentic RL with Tool-Augmented Verification

Agentic Reinforcement Learning with Tool Use (ARLT) enhances domain-specific performance in the LLM post-training stage by allowing the model to interact with external tools and learn from verified feedback. In this setting, the policy LLM π_θ generates responses token by token, where τ_t denotes the token at step t and $\tau_{<t}$ denotes the preceding context. A complete generated response $\tau = (\tau_1, \dots, \tau_T)$, corresponding to a full program or solution, is evaluated by a verified reward function $R(\tau)$.

Group Relative Policy Optimization (GRPO) [35] is a widely used RL algorithm in recent ARLT systems [21, 34], and has demonstrated strong empirical performance since [8]. Given a group of G rollout trajectories $\{\tau_i\}_{i=1}^G$ sampled from the current policy, GRPO

optimizes the following clipped surrogate objective:

$$\mathcal{J}(\theta) = \frac{1}{G} \sum_{i=1}^G \frac{1}{|\tau_i|} \sum_{t=1}^{|\tau_i|} \min\left(r_t^i(\theta) \hat{A}_i, \text{clip}(r_t^i(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_i\right), \quad (1)$$

where

$$r_t^i(\theta) = \frac{\pi_\theta(\tau_t^i | \tau_{<t}^i)}{\pi_{\text{ref}}(\tau_t^i | \tau_{<t}^i)}$$

is the token-level importance ratio with respect to a reference policy π_{ref} . The advantage $\hat{A}_i$ is computed via group-wise normalization:

$$\hat{A}_i = \frac{R(\tau_i) - \mu}{\sigma}, \quad \mu = \frac{1}{G} \sum_{j=1}^G R(\tau_j), \quad \sigma = \text{std}(\{R(\tau_j)\}_{j=1}^G).$$

In our setting, rewards are terminal—i.e., computed once after a complete OpenQASM program is generated rather than at intermediate token steps—and the same advantage $\hat{A}_i$ is assigned to all tokens within trajectory τ_i .

2.2 Target Representation: OpenQASM

OpenQASM (Open Quantum Assembly Language) [7] is a low-level programming language designed for expressing quantum circuits and operations, which is similar to traditional Hardware Description Language (HDL) like Verilog and VHDL. It serves as an intermediate representation (IR) for quantum algorithms, allowing them to be executed on various quantum hardware platforms. OpenQASM provides a standardized way to describe quantum gates, measurements, and other operations, making it easier for developers to write and share quantum programs.

OpenQASM is widely used in the quantum computing community, since many popular quantum computing software frameworks, such as IBM’s Qiskit [19], Google’s Cirq [9], Microsoft’s QDK [29], and Rigetti’s Forest[36], support OpenQASM as a means of serializing quantum circuits in a standard way. Developers write quantum algorithms in a high-level language and then compile them down to OpenQASM for distribution between different quantum hardware backends. OpenQASM has become the conjunction between quantum software and hardware, being critical for both platform-agnostic and platform-specific optimization, mapping, scheduling, evaluation, profiling, and simulation.

2.3 Quantum Application Context: Optimization Circuits

Quantum computing exploits the principles of quantum mechanics to perform computations using quantum bits (qubits) instead of classical bits. A qubit is a unit vector in a two-dimensional Hilbert space and can exist in a superposition $\alpha|0\rangle + \beta|1\rangle$ with $\alpha, \beta \in \mathbb{C}$ and $|\alpha|^2 + |\beta|^2 = 1$ [31]. Multi-qubit states arise via tensor products, and computation proceeds by applying unitary gates. Rotation gates $R_x(\theta)$, $R_y(\theta)$, $R_z(\theta)$ are unitary for all θ , and together with entangling operations (e.g., CNOT) and standard gates (e.g., Hadamard, CZ), they form a universal gate set [41]. By combining these gates into parameterized circuits $U(\theta)$, one obtains a flexible framework for variational quantum algorithms.

One of the most promising applications is quantum optimization [1]. Classical problems such as QUBO and HUBO [5, 28] can be rewritten in terms of spin variables and mapped to Pauli-Z operators, turning them into Hamiltonian minimization tasks. The goal is to prepare a parameterized state $U(\theta)|0\rangle$ whose expectation value $\langle 0|U^\dagger(\theta)HU(\theta)|0\rangle$ approximates the ground state of H . Hybrid quantum-classical methods iteratively optimize θ : QAOA uses a cost Hamiltonian and a mixer ansatz [12], VQE employs expressive or hardware-efficient ansatzes [33], and adaptive VQE constructs circuits gate by gate from a predefined pool [14].

3 VISTA: Agentic System Design

3.1 System Pipeline for Tool-Augmented RL Post-Training

End-to-End Workflow. Figure 3 illustrates the overall design of VISTA, which treats natural-language problem descriptions as high-level specifications and translates them into executable quantum circuits in OpenQASM through verified reinforcement learning.

Our training framework is built on Verl [40] and therefore follows its prompt structure. Each training input consists of two components: (1) a system message that defines the model’s role as a quantum circuit design assistant and provides instructions for generating OpenQASM circuits for quantum optimization tasks; and (2) a user message that specifies the problem description, the number of qubits, and any additional constraints.

During training, (A) the LLM actor receives the input prompt and generates a batch of OpenQASM circuits; (B) the generated circuits are encapsulated into a batch of tool calls; (C) these tool calls are dispatched to the Quantum Tool Server for execution and reward evaluation; (D) the server simulates each circuit, evaluates its performance on the target quantum task, and computes rewards according to predefined criteria; and (E) the rewards are processed through group-based computations to obtain advantages, which are then used to update the LLM actor via the GRPO algorithm. In our setup, a *group* consists of multiple quantum circuit samples generated from the same input prompt, enabling VISTA to compare the relative performance of alternative solutions to the same quantum task.

Online Interaction with a Quantum Agent. Building on Jiang et al. [21], our quantum tool server (quantum agent) consolidates environment interaction and reward computation into a single tool module (see Figure 3). At each RL step (C), the quantum agent receives a batch of tool calls via HTTP and executes specialized reward calculations for the generated quantum circuits.

3.2 Hierarchical Verified Reward Design

Reward rationale and theoretical motivation. For quantum optimization tasks, the objective is to produce a circuit whose measurement distribution concentrates on low-energy bitstrings, equivalently minimizing the expected energy:

$$J(C) := \langle \psi_C | H | \psi_C \rangle = \sum_z p_C(z) E(z).$$

Here, C denotes a quantum circuit that prepares the state $|\psi_C\rangle = U_C|0\rangle^{\otimes n}$, and $p_C(z) = |\langle z | \psi_C \rangle|^2$ is the distribution obtained by measuring $|\psi_C\rangle$ in the computational basis. For the optimization problems considered in this work (e.g., QUBO and Ising-type graph problems), the cost Hamiltonian H is diagonal in the computational basis and can be written as

$$H = \sum_{z \in \{0,1\}^n} E(z) |z\rangle\langle z|,$$

where $E(z)$ is the classical objective value associated with bitstring z .

Lemma 1 (Distributional alignment bounds energy error). Let p_C and p_{C^*} denote the measurement distributions induced by a generated circuit C and a reference circuit C^* , respectively. If H is diagonal in the computational basis, then

$$|J(C) - J(C^*)| \leq \|E\|_\infty \|p_C - p_{C^*}\|_1 \leq \|E\|_\infty \sqrt{2 \ln 2 \cdot \text{JS}(p_C \| p_{C^*})},$$

where $\|E\|_\infty = \max_z |E(z)|$.

Interpretation. Reducing the Jensen–Shannon divergence between measurement distributions bounds the deviation in expected energy, which motivates distributional alignment as a low-variance surrogate for the optimization objective.

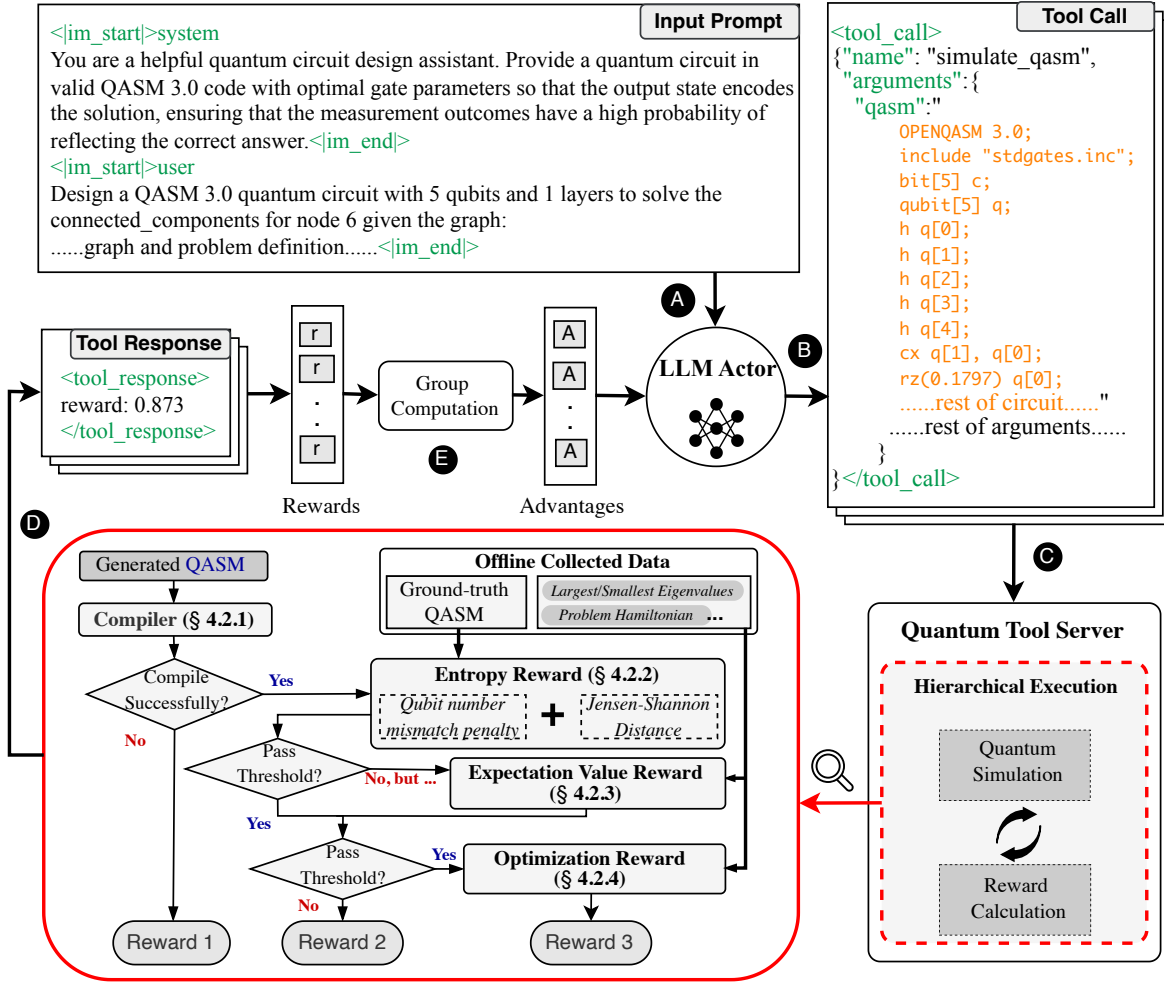


Fig. 3: VISTA design overview.

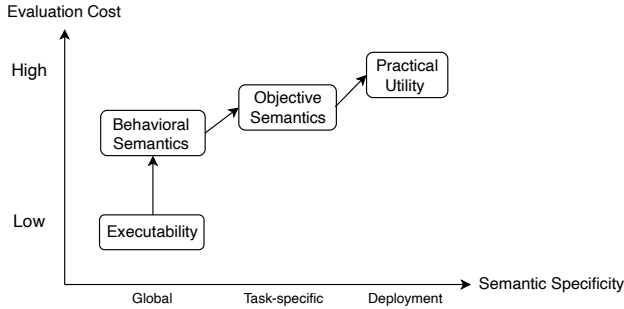


Fig. 4: Reward design space for quantum circuit generation. The semantic specificity axis reflects increasing contextual grounding of evaluation signals, from global behavior checks, through Hamiltonian-defined task objectives, to deployment-oriented algorithmic utility.

Formal reward definition. For each generated trajectory τ_i , corresponding to a complete OpenQASM program, we define a scalar terminal reward $R(\tau_i)$ as

$$R(\tau_i) = \begin{cases} -1, & R_{\text{syn}}(\tau_i) = -1, \\ w_2 s_2(\tau_i), & R_{\text{syn}}(\tau_i) = 1 \wedge \neg \text{Prom}(\tau_i), \\ w_2 s_2(\tau_i) + w_3 s_3(\tau_i) + w_4 R_{\text{opt}}(\tau_i), & R_{\text{syn}}(\tau_i) = 1 \wedge \text{Prom}(\tau_i). \end{cases} \quad (2)$$

where $R_{\text{syn}}(\tau_i) \in \{-1, 1\}$ is the syntactic validity reward, equal to 1 if the generated OpenQASM program parses successfully and -1 otherwise. The term $s_2(\tau_i)$ denotes the distributional alignment score based on the Jensen-Shannon distance, augmented with the qubit-mismatch penalty. The term $s_3(\tau_i)$ denotes the expectation-value score (defined in Equation 5), and $R_{\text{opt}}(\tau_i)$ denotes the optimization-progress reward (defined in Equation 7). The weights $w_2, w_3, w_4 \geq 0$ control the relative contribution of each reward component.

A trajectory τ_i is considered *promising* if

$$\text{Promising}(\tau_i) \iff (s_2(\tau_i) \geq \delta_{\text{JS}}) \vee (s_3(\tau_i) \geq \delta_{\text{EV}}),$$

where $\delta_{\text{JS}}, \delta_{\text{EV}} \in (0, 1)$ are predefined thresholds.

Integration with GRPO. Following the GRPO formulation introduced in §2, for a given input prompt x , we sample a group of G

trajectories $\{\tau_i\}_{i=1}^G$ from the current policy $\pi_\theta(\cdot | x)$. Each trajectory τ_i receives a terminal reward $R(\tau_i)$ defined in Equation 2. We compute the group-normalized advantage as

$$\hat{A}_i = \frac{R(\tau_i) - \mu}{\sigma}, \quad \mu = \frac{1}{G} \sum_{j=1}^G R(\tau_j), \quad \sigma = \sqrt{\frac{1}{G} \sum_{j=1}^G (R(\tau_j) - \mu)^2 + \epsilon}. \quad (2)$$

The policy is updated using the GRPO clipped surrogate objective. Let $a_{i,t}$ denote the t -th token in trajectory τ_i , and define the importance ratio

$$r_{i,t}(\theta) = \frac{\pi_\theta(a_{i,t} | a_{i,<t}, x)}{\pi_{\text{ref}}(a_{i,t} | a_{i,<t}, x)}.$$

Since the reward in Equation 2 is terminal, the same advantage $\hat{A}_i$ is assigned to all tokens in τ_i . The resulting GRPO objective is

$$\mathcal{L}_{\text{GRPO}}(\theta) = \mathbb{E}_x \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|\tau_i|} \sum_{t=1}^{|\tau_i|} \min \left(r_{i,t}(\theta) \hat{A}_i, \text{clip}(r_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_i \right) \right]. \quad (3)$$

In summary, the hierarchical quantum-aware reward in Equation 2 provides a verified scalar signal for each generated OpenQASM program. This signal is normalized within each prompt-level group and directly optimized via the GRPO objective in Equation 3.

Hierarchical reward mechanism. We design a hierarchical reward with four components: (i) *syntactic correctness*, (ii) *distributional alignment* (an entropy-based term), (iii) an *expectation-value* term, and (iv) an *optimization-progress* term. The computation proceeds hierarchically. We first verify syntactic correctness; if the generated OpenQASM is valid, we then measure the distributional discrepancy between the generated and ground-truth circuits (e.g., via the Jensen–Shannon distance) to quantify overall mismatch. Generated circuits may realize unitaries $\hat{U}$ that deviate substantially from the ground truth U^* , leading to different measurement distributions in the computational basis. However, for a given quantum optimization task, performance can remain comparable because evaluation depends on the problem Hamiltonian H through $E(\psi) = \langle \psi | H | \psi \rangle$. To capture this task-specific behavior, we further incorporate two problem-aware terms: (a) an *expectation-value reward* that evaluates Hamiltonian-based performance, and (b) an *optimization-progress reward* that credits the improvement achieved by a local optimizer. Overall, the reward reflects both the steps required to improve the generated circuit and the gap between the converged solution and the reference.

In addition, we introduce a *qubit-mismatch penalty* in stage (ii) to discourage generating an inconsistent number of qubits. This penalty addresses a common failure mode in our training loop: the LLM may output a circuit whose qubit count differs from the target, which can lead to dimension mismatch and reward evaluation errors (e.g., generating qubit[7] q; for a 9-qubit prompt).

We adopt a gated procedure to compute rewards, as illustrated in the red line box of Figure 3. If the distributional divergence between the generated and ground-truth circuits exceeds a threshold, we additionally evaluate the expectation-value reward. If either the (normalized) expectation-value score surpasses its threshold or

the distributional divergence falls below its threshold, we run a local optimizer and assign an optimization-progress reward. This design avoids expensive evaluations for clearly low-quality circuits while remaining robust to cases where distributions differ but task performance is comparable. In the following, we introduce each reward component in detail.

3.2.1 Syntactic reward. The syntactic reward captures whether the generated OpenQASM program is syntactically valid. A circuit is considered syntactically correct if it can be parsed by the Qiskit OpenQASM 3.0 parser, implying that it follows the grammar of the OpenQASM standard [7]. If parsing fails, reward computation terminates and returns -1 .

3.2.2 Entropy reward. Prior work [20] evaluated circuit quality using relative entropy, i.e., Kullback–Leibler (KL) divergence. In this work, we use the Jensen–Shannon (JS) distance, which yields a bounded value in $[0, 1]$ and is more stable for RL training. The Jensen–Shannon distance is defined as

$$d_{\text{JS}}(p, q) = \sqrt{\frac{\text{JS}(p \| q)}{\log 2}}, \quad (4)$$

where

$$\text{JS}(p \| q) = \frac{1}{2} D_{\text{KL}}(p \| m) + \frac{1}{2} D_{\text{KL}}(q \| m),$$

$m = \frac{1}{2}(p + q)$, and D_{KL} is the KL divergence. We define the reward as $1 - d_{\text{JS}}(p, q) \in [0, 1]$, where 1 indicates identical distributions and 0 indicates highly dissimilar distributions. By Lemma 1, for diagonal cost Hamiltonians, reducing JS divergence also bounds the deviation in expected energy, making this term an effective surrogate for task performance.

Qubit-mismatch penalty. In practice, the generated QASM may specify a different number of qubits than the ground-truth QASM, making the entropy-based reward inapplicable due to a dimension mismatch. Let n_{gen} and n_{gt} be the qubit counts of the generated and ground-truth circuits, and let $k = \min(n_{\text{gen}}, n_{\text{gt}})$. We define a qubit-mismatch penalty

$$R_{\text{qm}} = \text{clip}_{[-0.2, 0]} (\alpha + \beta \Delta n + \gamma a_{\text{extra}} + \eta e_{\text{cross}}), \quad \Delta n = |n_{\text{gen}} - n_{\text{gt}}|,$$

where a_{extra} counts *active* extra qubits beyond the first k (idle or reset-only ancillas incur little or no penalty), and e_{cross} counts multi-qubit gates that entangle extra wires with core wires. To ensure fairness, we compute d_{JS} on the first k qubits (via marginalization) and down-scale its weight by the mismatch severity.

For the expectation-value and optimization-progress terms below, we pad the problem Hamiltonian with identities so that its width matches the circuit, preserving comparability while preventing reward hacking through ancillary qubits.

3.2.3 Expectation-value reward. While distributional alignment controls energy error relative to a specific reference distribution, multiple optimal or near-optimal solutions may induce different distributions. The expectation-value reward therefore directly measures normalized suboptimality with respect to the Hamiltonian.

Our training dataset consists of optimization problems expressed as eigenvalue minimization tasks. For an ideal circuit, measurement would return the optimal eigenvalue with probability 1, and the expectation value would coincide with the optimum. In practice, the

measured expectation value is typically larger. We therefore define a reward based on the expectation value of the cost Hamiltonian.

For a syntactically correct circuit, we simulate the circuit and compute the expectation value of the problem-specific cost Hamiltonian, denoted by E_{gen} . We also compute the minimum and maximum eigenvalues of the Hamiltonian, denoted by E_{min} and E_{max} , respectively. Since $0 \leq E_{\text{min}} < E_{\text{gen}} < E_{\text{max}}$, we apply min-max normalization:

$$f_{\text{max}}^{\text{min}}(E_{\text{gen}}) := \frac{E_{\text{gen}} - E_{\text{min}}}{E_{\text{max}} - E_{\text{min}}}, \quad (5)$$

which yields a value in $[0, 1]$. We define the expectation-value reward as

$$s_3(\tau_i) := 1 - f_{\text{max}}^{\text{min}}(E_{\text{gen}}) = 1 - \frac{E_{\text{gen}} - E_{\text{min}}}{E_{\text{max}} - E_{\text{min}}}, \quad (6)$$

3.2.4 Optimization reward. It is unlikely that LLMs generate parameterized quantum circuits that immediately solve the given optimization problems. A practical workflow therefore includes an additional phase where the user continues optimizing the parameters of the generated circuit. Given the difficulty of parameter optimization, a key indicator of usefulness is the number of optimization steps required to reach a sufficiently low expectation value.

We define an optimization-progress reward as $1/(1+n)$, where n is the number of optimization steps required for the generated circuit to reach an optimized circuit for the given problem. In addition, we favor circuits that achieve lower final expectation values. Let E_{opt} denote the expectation value after local optimization, and let E_{min} again be the optimal eigenvalue. We therefore use

$$R_{\text{opt}}(\tau_i) := \frac{1}{1+n} + \left(1 - f_{\text{max}}^{\text{min}}(E_{\text{opt}})\right), \quad (7)$$

where $f_{\text{max}}^{\text{min}}$ is defined in Equation 5. Under standard local convergence assumptions of classical optimizers, the number of optimization steps required to reach a target energy threshold is monotone in the initial suboptimality, making optimization progress a proxy for initialization quality.

4 Evaluation

4.1 Evaluation Setup

4.1.1 Training Setup.

Training Data. We utilize the training data from [20], which is one of the most extensive available datasets of quantum circuits in QASM format covering 12 central optimization problem primitives on graphs [22], and many of their abstract descriptions appeared in [28]. The full training dataset contains 13,914 samples, which we split into training and validation sets using a 96/4 ratio. The key characteristics in this dataset are the parametrized circuits with optimal parameters, the problem Hamiltonians, and the smallest and largest eigenvalues for the Hamiltonians. The full dataset is constructed so that for each optimization problem, the system generates a random graph, where the problem is solved using QAOA, VQE, and adaptive VQE. If the quantum optimization problem is simulable and optimization converges, the problem, graph, circuits in QASM format, and Hamiltonian, along with other data, are included in the training dataset. To show VISTA is able to handle

Tab. 1: Main training settings for agentic RL with a 4B SFT model.

Component	Configuration
Base model	Qwen3-4B-Instruct-2507
SFT setup	Quantum Datasets [20]
SFT learning rate	2×10^{-5}
Rollout Num	16
Temperature	0.7
top_p	0.8
top_k	-1
Token limits	1024 (Prompt), 9216 (Response), 2048 (Observation), 8192 (Action)
Optimizer	AdamW [27]
Learning rate	1×10^{-6}
Epochs	10
Batch size	128

large scale of quantum circuits, we also provide the statistics of qubit number, circuit depth, gate number, and number of layers of quantum circuits in Figure 5. The maximum number of qubits reaches 16, maximum number of gates is 1138, and the maximum circuit depth is 839.

The offline collected ground-truth QASM circuits are passed as reward references, and other information such as Hamiltonians and eigenvalues are also included as extra information for reward computation, which will all be passed to quantum simulator server through HTTP in every training loop.

Training Model. We fine-tune a 4B model with GRPO on 16×H100-64GB GPUs using FSDP [42]. Each prompt samples $n=16$ rollouts via vLLM [23] (temperature 0.7, top- $p = 0.8$). The average training time is 48 hours.

The 4B model is supervised fine-tuned from Qwen3-4B-Instruct-2027 with the method of Agent-Q[20].

Evaluation Metrics. We evaluate the fine-tuned model across four complementary metrics designed to assess both syntactic fidelity and optimization quality. First, we measure **syntactical correctness ratio (SCR)**, defined as the percentage of generated outputs that can be parsed as valid OpenQASM 3.0 circuits using Qiskit’s QASM parser. This metric captures whether the model has internalized the grammar of the domain-specific language. Second, we perform **successful rate of expectation value (SREV)**, where each syntactically correct circuit is simulated and the expectation value for the problem-specific cost Hamiltonian H is computed. We compute the expectation value of generated circuit C as $E(C) = \langle \psi_C | H | \psi_C \rangle$. Let E^* denote the ground-truth QASM’s expectation. The circuit is counted as *successful* if $|E(C) - E^*| \leq 0.2$, and SREV is the percentage of successful QASMs. Third, we evaluate the **relative entropy (RE) of probability distributions** by computing the KL divergence $D_{\text{KL}}(P_{\text{sol}} \parallel P_{\text{gen}})$ between the outcome distribution of the generated circuit and that of the optimized reference. Finally, we compute the **High-Quality Circuit Ratio (HQCR)**, defined as the proportion of generated circuits whose relative entropy against the ground-truth distributions is within a threshold 0.1. This metric provides a more interpretable measure of how often the model produces reasonable solutions. Each metric is measured in Pass@1 and Pass@10, where Pass@1 evaluates whether a single sampled QASM per prompt

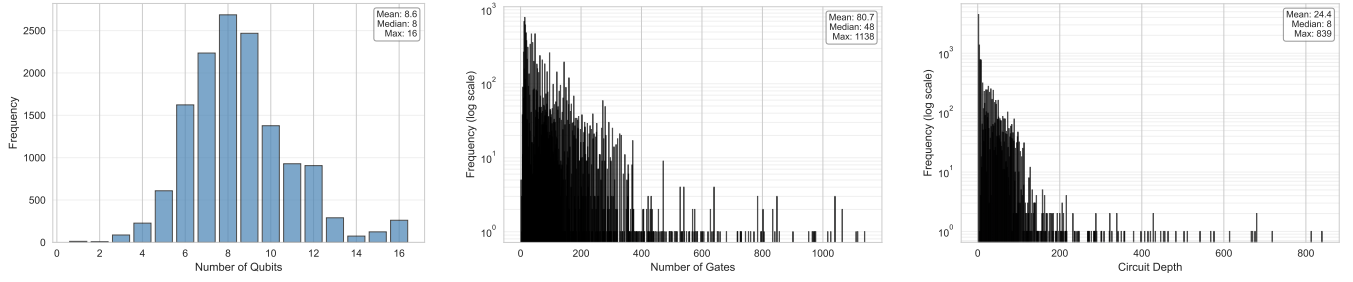


Fig. 5: Distribution of (a) number of qubits, (b) number of gates, and (c) circuit depth in the dataset.

meets the criterion, and Pass@10 evaluates whether at least one out of ten sampled QASMs meets the criterion. Results are shown in Table 2, where up/down arrows by column names indicate whether higher or lower values are better.

Baselines. The baselines can be categorized into three groups. (i) **Prompting-Based State-of-the-art LLMs:** We evaluate DeepSeek-V3 [26] and OpenAI’s GPT-4o [16] and GPT-5 [32] via their official API, with GPT-5 being OpenAI’s latest flagship. All models are evaluated using few-shot prompting with four demonstration examples. (ii) **SFT-only:** We train Qwen-3-4B using SFT only. (iii) **RL-only:** We evaluate a cold-start model trained using the GRPO algorithm [35].

4.2 Main Results

Tab. 2: VISTA’s Pass@K and comparison with existing techniques.

Methods	Pass@1				Pass@10			
	SCR ↑	SREV ↑	RE ↓	HQCR ↑	SCR ↑	SREV ↑	RE ↓	HQCR ↑
Few-Shot Prompting								
DeepSeek-V3	94.83%	12.24%	19.20	10.00%	98.97%	26.38%	16.39	16.38%
GPT-5	87.07%	10.00%	19.94	6.90%	90.52%	27.07%	11.57	16.55%
GPT-4o	87.93%	9.83%	19.42	6.38%	88.79%	18.62%	14.08	12.07%
Post-Training (Qwen3-4B)								
Agent-Q	97.41%	18.97%	12.74	15.17%	99.65%	31.55%	10.81	23.62%
Cold Start GRPO	84.48%	19.84%	14.32	12.41%	95.17%	27.59%	11.38	18.96%
VISTA (ours)	99.31%	22.41%	11.61	17.24%	100%	33.10%	8.48	27.24%

Performance Analysis. We report the performance of VISTA in Table 2. VISTA achieves strong results against all baselines, with a pass@1 of 99.31% SCR. Compared to the best competing methods, it yields a +12.95% improvement in SREV (22.41 vs. 19.41 for Cold Start (GRPO)), an +8.87% reduction in RE (11.61 vs. 12.74 for Agent-Q), and a +13.64% gain in HQCR over Agent-Q. For pass@10, we even achieve 100% syntactical correctness and even better semantic improvement.

Meanwhile, as noted earlier, HQCR was measured with a fixed threshold of 0.1. In Figure 6, we vary this threshold from 0.1 to 0.9 and report The fraction of valid QASMs that qualify as high-quality. At best, VISTA achieves a +13.65% improvement over Agent-Q, +72.4% over DeepSeek-V3, and up to 1.65× and 1.50× improvements over GPT-4o and GPT-5, respectively. Furthermore, to quantify the semantic closeness of the generated QASMs to the ground

truth, we also measure $\Delta E = |E(C) - E^*|$ defined before; note that smaller values indicate higher-quality QASMs. Figure 7 illustrates the distribution of ΔE for each model, VISTA achieves substantial improvements over all baselines, including a 4.9% reduction in the median ΔE and a 9.7% reduction in the upper quartile compared to the second-best Agent-Q.

In addition, we compare VISTA-generated QASMs with a *random parameter initialization baseline*, a common practice in hybrid quantum-classical algorithms. For each VISTA circuit, we evaluated JS-divergence and expectation value relative to the Hamiltonian and ground truth, alongside 100 randomized variants with parameters sampled from $(-\pi, \pi]$. The results show that VISTA consistently outperforms random initialization, achieving lower JS-divergence (0.79 vs. 0.95) and expectation values closer to the optimum (0.16 vs. 0.36).

4.3 Ablation: Hierarchical Reward

We quantify the contribution of each reward term by independently removing each component of the hierarchy reward in section 3. For each variant, we disable exactly one component while keeping the remainder unchanged; we also report a *Validity-only* sanity baseline, which only returns reward based on whether the generated QASM is valid(1) or not(-1). All runs are initialized from the same SFT checkpoint and trained with matched compute (identical optimizer, schedule, batch size, steps, and decoding settings). The evaluation metrics are the same as presented in section 3, and the results are summarized in Table 3.

Distributional alignment (RE) is the primary driver of all metrics. Removing RE causes the largest drop across semantic metrics and, unexpectedly, even harms syntactic correctness. This highlights that coarse-grained alignment of measurement distributions is essential for effective reward shaping. **Expectation value (EV) safeguards hard cases.** Disabling EV reduces SREV as expected and slightly degrades other semantic metrics. When distributional divergence is high, EV provides a task-specific signal that rescues otherwise borderline QASMs. **Optimization progress (Opt) provides incremental gains.** Removing Opt leads to a notable drop in HQCR, with a larger gap at Pass@10 than at Pass@1. This suggests that rewarding fewer optimization steps can also benefit the training. **Qubit mismatch penalty (QMP) ensures stability.** Without QMP, qubit-count inconsistencies increase and reward noise emerges (e.g., evaluation failures or padded comparisons), lowering all metrics. **Validity-only rewards are insufficient.** A reward that enforces only basic validity achieves reasonable SCR

but lags considerably on SREV, RE, and HQCR, even performing slightly worse than Agent-Q. This underscores the necessity of semantically informed reward signals.

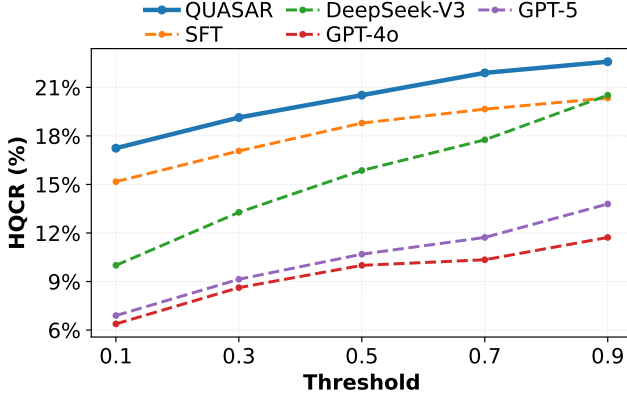


Fig. 6: HQCR with varying threshold.

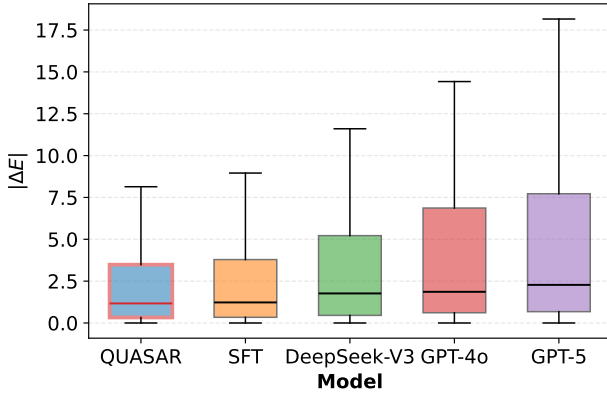


Fig. 7: ΔE for valid QASMs.

4.4 Scalability Analysis

In this section, we examine the scalability of VISTA with respect to the number of qubits, which directly determines the size of the underlying Hilbert space and the computational cost of quantum verification through simulation. We analyze how both the syntactic and semantic correctness of the generated quantum circuits evolve as the problem size increases.

As shown in Figure 8, the qubit count has little impact on the syntactic correctness of VISTA and Agent-Q. In contrast, when the number of qubits reaches 14, industrial LLMs with few-shot prompting exhibit a sharp increase in compilation errors. Furthermore, Figure 9 indicates that the semantic correctness (HQCR) of industrial LLMs with few-shot learning degrades rapidly, dropping to nearly zero at larger qubit sizes.

Across both metrics, VISTA consistently outperforms all baselines, demonstrating strong robustness as the problem scale increases.

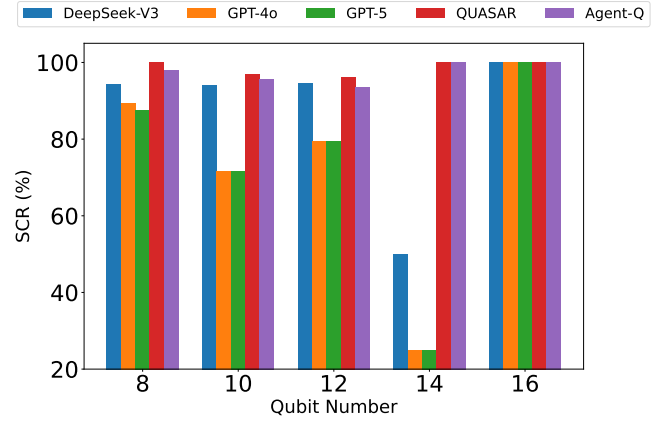


Fig. 8: SCR with varying qubit number.

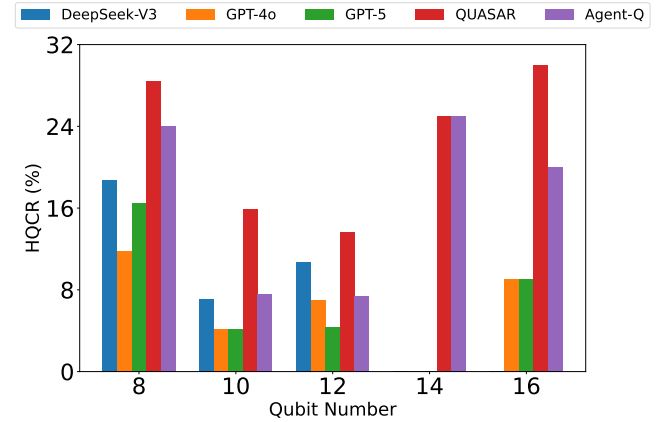


Fig. 9: HQCR with varying qubit number.

4.5 Real-Device Deployment on a Hybrid Quantum-Classical Stack

This section evaluates whether VISTA-generated circuits remain semantically correct when executed on a real quantum device, where noise, connectivity constraints, and compiler choices can materially change the effective computation.

Goal and comparison protocol. For each problem instance, we compare (i) the ground-truth optimized reference circuit from the dataset, and (ii) the VISTA-generated circuit, after mapping both onto the same target device using the same compilation pipeline and evaluation budget. We report both *algorithmic quality* (energy / solution probability) and *hardware cost* (depth, two-qubit gates, runtime proxies).

4.5.1 Hardware and software stack.

Target device and calibration snapshot. Specify: device name, qubit count, coupling map, native gate set, and the calibration time window (e.g., a fixed daily snapshot). For fair comparison, execute all methods within the same calibration window.

Hybrid stack. Describe the orchestration path (controller → simulator/reward server → device backend), and how job submission, result retrieval, and post-processing are handled.

Tab. 3: Reward ablation for VISTA: contribution of each component.

Variant	Components	Pass@1				Pass@10			
		SCR ↑	SREV ↑	RE ↓	HQCR ↑	SCR ↑	SREV ↑	RE ↓	HQCR ↑
Full (Vista)	Val + RE + EV + Opt + QMP	99.31%	22.41%	11.61	17.24%	100%	33.10%	8.48	27.24%
w/o EV term	Val + RE + Opt + QMP	98.62%	20.69%	11.82	16.38%	100%	31.03%	9.12	26.72%
w/o RE term	Val + EV + Opt + QMP	66.38%	5.17%	24.67	5.69%	79.82%	15.52%	18.26	16.90%
w/o Opt term	Val + RE + EV + QMP	98.79%	21.90%	11.98	16.90%	100%	32.76%	9.01	26.55%
w/o QMP	Val + RE + EV + Opt	98.79%	21.72%	12.02	16.21%	100%	31.55%	9.48	27.06%
Validity only	Val	99.13%	18.79%	12.89	14.66%	100%	30.86%	11.27	23.27%

4.5.2 Workloads and circuit selection.

Benchmarks. Use the same primitives as in training/evaluation (e.g., MaxCut, MIS, etc.), but select instances that fit the device constraints.

Instance filters. Report and enforce: (1) maximum qubits used on hardware (e.g., 8–16), (2) maximum compiled depth, and (3) maximum two-qubit gate count. If a circuit cannot be mapped under these constraints, count it as a failure and report the rate.

Circuit sets. For each instance, include: (i) ground-truth QASM circuit, (ii) VISTA-generated QASM circuit (Pass@1 or Pass@10 best-of selection; state which), (iii) a random-parameter baseline on the same ansatz (as in offline analysis), and optionally (iv) an SFT-only and/or RL-only ablation if available.

4.5.3 Compilation and execution.

Transpilation policy (fixed). Use one fixed transpilation configuration for all methods: optimization level, routing method, layout strategy, and any pulse-level options if applicable. Report the compiled metrics (depth, two-qubit gates, total gates) for each method.

Measurement protocol. Execute each (compiled) circuit with a fixed number of shots S (e.g., 4k–10k), and repeat each circuit R times (e.g., 3–5) to estimate variability. Randomize the job order to reduce drift bias.

Observable estimation. Compute the cost Hamiltonian expectation $E(C)$ from measurement outcomes using the same Hamiltonian definition as offline evaluation. If the Hamiltonian is a sum of Pauli terms, explicitly state the measurement grouping strategy (or note that only Z -diagonal costs are used).

4.5.4 Metrics on hardware. We recommend reporting the following metrics per benchmark and overall.

Energy gap on hardware. $\Delta E_{hw} = |E_{hw}(C) - E_{hw}(C^*)|$.

Solution quality. Probability mass on optimal/near-optimal bit-strings (when classically computable for small n), or best-found objective value from samples.

Distributional similarity. KL or JS divergence between the (hardware) outcome distributions of VISTA vs. reference.

Robustness. Mean±std across repeats; optionally the worst-case over repeats.

Hardware efficiency. Compiled depth and two-qubit gate count; optionally an aggregate cost like depth×2Q-gates.

4.5.5 Noise handling and mitigation.

Fig. 10: Hybrid quantum–classical stack used for real-device evaluation.

No mitigation (base). First report raw (unmitigated) results.

Optional mitigation (ablations). If you apply mitigation, include it as an ablation, e.g., measurement error mitigation and/or zero-noise extrapolation. Apply the same mitigation procedure to all methods.

4.5.6 Suggested organization and tables/figures.

Recommended write-up order. (1) Stack and device, (2) workloads/filters, (3) compilation + execution protocol, (4) metrics, (5) main results, (6) mitigation ablations, (7) failure analysis (unmapable circuits, depth blow-ups, unstable runs).

Main results table (template). Report per method: valid-on-hardware rate, ΔE_{hw} , solution probability, JS/KL, compiled depth, and 2Q gate count.

5 Related Work

In recent years, there has been increasing interest in applying GPT- and LLM-based tools to address challenges in designing and constructing circuits for various quantum computational challenges. In one of the first works, [17] fine-tuned large language models to serve as a Qiskit Code Assistant, which was evaluated with [39]. This work was further improved with post-training reinforcement learning [10]. [6] leveraged Chain-of-Thought (CoT) reasoning and Retrieval-Augmented Generation (RAG) in a multi-agent setting to facilitate the synthesis of quantum circuits. These efforts are closely tied to the Qiskit framework, whereas our approach targets OpenQASM 3.0 [7], a platform-independent standard now supported by Qiskit, PennyLane, and Cirq. LLMs have also been used for optimizing ansatz design [25, 38], and [4] used language models to design quantum experiments. [15] utilized LLMs for guided ansatz design in financial modeling.

Compared to LLM-based methods, standard transformers and GPT-based systems have received more attention. [13] trained the standard GPT model to predict measurement outcomes from a neutral atom quantum computer. The findings revealed limitations in the standard GPT model’s ability to predict measurement outcomes, which could prove valuable in expanding our knowledge of the boundaries of present-day LLMs. Using the standard transformer-based approach, Nvidia has designed an optimization pipeline that produces quantum circuits, which are aimed at identifying ground states in electronic structure Hamiltonians [30]. Since the trainable parameters are in the transformer model, the method tries to circumvent some of the problems in current variational methods, like

barren plateaus. [3] developed KetGPT, which uses a GPT-based model to generate realistic quantum circuits. The model is trained in QASMBench circuits [24]. The circuits are restricted to the OpenQASM 2.0 format, without supporting parameters. Finally, [37] developed QAOA-GPT, which is capable of generating QAOA-type circuits that solve the MaxCut problem. [11] is the first to use a quantum-verifiable reward to post-training LLMs to generate Qiskit code.

6 Conclusion

We presented VISTA, a verifier-in-the-loop, tool-augmented agentic RL system for semantic program synthesis, with OpenQASM 3.0 quantum circuit generation as a successful application. The paper makes three core contributions. First, we contribute the core system: verifier-in-the-loop, tool-augmented agentic RL system for semantic program synthesis. Second, we contribute optimization technique I: hierarchical verified reward optimization, which provides stable, task-aligned learning signals from executability through objective-level quality. Third, we contribute optimization technique II: cost-aware gated evaluation optimization, which reduces verifier overhead by prioritizing low-cost checks and reserving expensive evaluations for promising candidates.

Empirically, VISTA consistently outperforms prompting-based industrial LLMs and SFT-only and RL-only baselines on both executability and semantic quality. Overall, the results show that combining hierarchical reward shaping with gated verification is an effective optimization strategy for AI agent systems in semantics-heavy generation tasks, with quantum software as a successful application domain.

References

- [1] Amira Abbas, Andris Ambainis, Brandon Augustino, Andreas Bärttschi, Harry Buhman, Carleton Coffrin, Giorgio Cortiana, et al. 2024. Challenges and opportunities in quantum optimization. *Nature Reviews Physics* 6, 12 (Dec. 2024), 718–735. doi:10.1038/s42254-024-00770-9 Publisher: Nature Publishing Group.
- [2] Amazon Web Services. [n. d.]. Amazon Braket. <https://aws.amazon.com/braket/>
- [3] Boran Apak, Medina Bandic, Aritra Sarkar, and Sebastian Feld. 2024. KetGPT – Dataset Augmentation of Quantum Circuits Using Transformers. In *Computational Science – ICCS 2024*, Leonardo Franco, Clélia de Mulatier, Maciej Paszynski, Valeria V. Krzhizhanovskaya, Jack J. Dongarra, and Peter M. A. Sloot (Eds.). Springer Nature Switzerland, Cham, 235–251.
- [4] Sören Arlt, Haonan Duan, Felix Li, Sang Michael Xie, Yuhuai Wu, and Mario Krenn. 2024. Meta-Designing Quantum Experiments with Language Models. *arXiv preprint arXiv:2406.02470* (2024).
- [5] Endre Boros and Peter L. Hammer. 2002. Pseudo-Boolean optimization. *Discrete Applied Mathematics* 123, 1 (2002), 155–225. doi:10.1016/S0166-218X(01)00341-9
- [6] Charlie Campbell, Hao Mark Chen, Wayne Luk, and Hongxiang Fan. 2025. Enhancing LLM-based Quantum Code Generation with Multi-Agent Optimization and Quantum Error Correction. *arXiv preprint arXiv:2504.14557* (2025).
- [7] Andrew W. Cross, Ali Javadi-Abhari, Thomas Alexander, Niel de Beaudrap, Lev S. Bishop, et al. 2022. OpenQASM 3: A broader and deeper quantum assembly language. *ACM Transactions on Quantum Computing* 3, 3 (Sept. 2022), 1–50. doi:10.1145/3505636 arXiv:2104.14722 [quant-ph].
- [8] DeepSeek-AI, Daya Guo, and et al. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv preprint arXiv:2501.12948* (2025). doi:10.48550/arXiv.2501.12948
- [9] Cirq Developers. 2025. Cirq. Zenodo. doi:10.5281/ZENODO.4062499
- [10] Nicolas Dupuis, Adarsh Tiwari, Youssef Mroueh, David Kremer, Ismael Faro, and Juan Cruz-Benito. 2025. Quantum Verifiable Rewards for Post-Training Qiskit Code Assistant. *arXiv:2508.20907* (Aug. 2025). doi:10.48550/arXiv.2508.20907 arXiv:2508.20907 [quant-ph].
- [11] Nicolas Dupuis, Adarsh Tiwari, Youssef Mroueh, David Kremer, Ismael Faro, and Juan Cruz-Benito. 2025. Quantum Verifiable Rewards for Post-Training Qiskit Code Assistant. *arXiv preprint arXiv:2508.20907* (2025).
- [12] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. 2014. A Quantum Approximate Optimization Algorithm. *arXiv preprint arXiv:1411.4028* (2014). doi:10.48550/arXiv.1411.4028
- [13] David Fitzek, Yi Hong Teoh, Hin Pok Fung, Gebremedhin A. Dagnew, Ejaaz Merali, M. Schuyler Moss, Benjamin MacLellan, and Roger G. Melko. 2024. RydbergGPT. *arXiv preprint arXiv:2405.21052* (may 2024). doi:10.48550/arXiv.2405.21052
- [14] Harper R. Grimsley, Sophia E. Economou, Edwin Barnes, and Nicholas J. Mayhall. 2019. An adaptive variational algorithm for exact molecular simulations on a quantum computer. *Nature Communications* 10, 1 (July 2019), 3007. doi:10.1038/s41467-019-10988-2
- [15] Yaswitha Gujju, Romain Harang, and Tetsuo Shibuya. 2025. LLM-Guided Ansätze Design for Quantum Circuit Born Machines in Financial Generative Modeling. *arXiv:2509.08385* [quant-ph]
- [16] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. 2024. GPT-4o System Card. *arXiv preprint arXiv:2410.21276* (2024).
- [17] IBM Quantum. 2025. Qiskit Code Assistant. <https://quantum.cloud.ibm.com/docs/en/guides/qiskit-code-assistant> Accessed: 2025-07-11.
- [18] Ali Javadi-Abhari, Matthew Treinish, Kevin Krsulich, Christopher J. Wood, Jake Lishman, Julien Gacon, Simon Martiel, Paul D. Nation, Lev S. Bishop, Andrew W. Cross, Blake R. Johnson, and Jay M. Gambetta. 2024. Quantum Computing with Qiskit. *arXiv preprint arXiv:2405.08810* (2024). doi:10.48550/arXiv.2405.08810
- [19] Ali Javadi-Abhari, Matthew Treinish, Kevin Krsulich, Christopher J. Wood, Jake Lishman, Julien Gacon, Simon Martiel, Paul D. Nation, Lev S. Bishop, Andrew W. Cross, Blake R. Johnson, and Jay M. Gambetta. 2024. Quantum computing with Qiskit. *arXiv:2405.08810* [quant-ph]
- [20] Linus Jern, Valter Uotila, Cong Yu, and Bo Zhao. 2025. Agent-Q: Fine-Tuning Large Language Models for Quantum Circuit Generation and Optimization. In *2025 IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE.
- [21] Dongfu Jiang, Yi Lu, Zhuofeng Li, Zhiheng Lyu, Ping Nie, Haozhe Wang, Alex Su, Hui Chen, Kai Zou, Chao Du, et al. 2025. VerTool: Towards Holistic Agentic Reinforcement Learning with Tool Use. *arXiv preprint arXiv:2509.01055* (2025).
- [22] Richard M. Karp. 1972. *Reducibility among Combinatorial Problems*. Springer US, Boston, MA, 85–103. doi:10.1007/978-1-4684-2001-2_9
- [23] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles* (Koblenz, Germany) (SOSP '23). Association for Computing Machinery, New York, NY, USA, 611–626. doi:10.1145/3600006.3613165
- [24] Ang Li, Samuel Stein, Sriram Krishnamoorthy, and James Ang. 2023. QASMBench: A Low-Level Quantum Benchmark Suite for NISQ Evaluation and Simulation. *ACM Transactions on Quantum Computing* 4, 2, Article 10 (Feb. 2023), 26 pages. doi:10.1145/3550488
- [25] Zhiding Liang, Jinglei Cheng, Rui Yang, Hang Ren, Zhixin Song, Di Wu, Xuehai Qian, Tongyang Li, and Yiyu Shi. 2023. Unleashing the Potential of LLMs for Quantum Computing: A Study in Quantum Architecture Design. *arXiv:2307.08191* (July 2023). doi:10.48550/arXiv.2307.08191 arXiv:2307.08191 [quant-ph].
- [26] Aixun Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. DeepSeek-V3 Technical Report. *arXiv preprint arXiv:2412.19437* (2024).
- [27] Ilya Loshchilov and Frank Hutter. 2017. Decoupled Weight Decay Regularization. *arXiv preprint arXiv:1711.05101* (2017).
- [28] Andrew Lucas. 2014. Ising formulations of many NP problems. *Frontiers in Physics* 2 (Feb. 2014). doi:10.3389/fphy.2014.00005
- [29] Microsoft. [n. d.]. *Azure Quantum Development Kit*. <https://github.com/microsoft/qsharp>
- [30] Kouhei Nakaji, Lasse Bjørn Kristensen, Jorge A. Campos-Gonzalez-Angulo, Mohammad Ghazi Vakili, Haozhe Huang, Mohsen Bagherimehrab, Christoph Gorgulla, FuTe Wong, Alex McCaskey, Jin-Sung Kim, Thien Nguyen, Pooja Rao, and Alan Aspuru-Guzik. 2024. The generative quantum eigensolver (GQE) and its application for ground state search. *arXiv preprint arXiv:2401.09253* (jan 2024). doi:10.48550/arXiv.2401.09253
- [31] Michael A. Nielsen and Isaac L. Chuang. 2010. *Quantum computation and quantum information* (10th anniversary ed ed.). Cambridge University Press, Cambridge ; New York.
- [32] OpenAI. 2025. *Introducing GPT-5*. <https://openai.com/index/introducing-gpt-5/> Accessed: 2025-09-21.
- [33] Alberto Peruzzo, Jarrod McClean, Peter Shadbolt, Man-Hong Yung, Xiao-Qi Zhou, Peter J. Love, Alan Aspuru-Guzik, and Jeremy L. O'Brien. 2014. A variational eigenvalue solver on a photonic quantum processor. *Nature Communications* 5 (July 2014), 4213. doi:10.1038/ncomms5213
- [34] Cheng Qian, Emre Can Acikgoz, Qi He, Hongru Wang, Xiushi Chen, Dilek Hakkani-Tür, Gokhan Tur, and Heng Ji. 2025. ToolRL: Reward is All Tool Learning Needs. *arXiv:2504.13958* [cs.LG]
- [35] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300* (2024).

- [36] Robert S. Smith, Michael J. Curtis, and William J. Zeng. 2017. A Practical Quantum Instruction Set Architecture. arXiv:1608.03355 [quant-ph]
- [37] Ilya Tyagin, Marwa H. Farag, Kyle Sherbert, Karunya Shirali, Yuri Alexeev, and Ilya Safro. 2025. QAOA-GPT: Efficient Generation of Adaptive and Regular Quantum Approximate Optimization Algorithm Circuits. arXiv:2504.16350 (April 2025). doi:10.48550/arXiv.2504.16350 arXiv:2504.16350 [quant-ph].
- [38] Kento Ueda and Atsushi Matsuo. 2025. Optimizing Ansatz Design in Quantum Generative Adversarial Networks Using Large Language Models. arXiv:2503.12884 (March 2025). doi:10.48550/arXiv.2503.12884 arXiv:2503.12884 [quant-ph].
- [39] Sanjay Vishwakarma, Francis Harkins, Siddharth Golecha, Vishal Sharathchandra Bajpe, Nicolas Dupuis, Luca Buratti, David Kremer, Ismael Faro, Ruchir Puri, and Juan Cruz-Benito. 2024. Qiskit HumanEval: An Evaluation Benchmark For Quantum Code Generative Models. In *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, Vol. 1. IEEE, 1169–1176.
- [40] volcengine. 2025. verl: Volcano Engine Reinforcement Learning for LLMs. <https://github.com/volcengine/verl>. Accessed: 2025-12-27.
- [41] Zebo Yang, Maede Zolanvari, and Raj Jain. 2023. A Survey of Important Issues in Quantum Computing and Communications. *IEEE Communications Surveys & Tutorials* 25, 2 (2023), 1059–1094. doi:10.1109/COMST.2023.3254481
- [42] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, et al. 2023. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel. arXiv:2304.11277 [cs.DC]